Ejercicio Servidor de Chat TCP Multicliente con Demultiplexación

Desarrolla un sistema cliente-servidor donde:

- El servidor acepte múltiples conexiones TCP.
- Cada cliente se atienda en un thread independiente.
- El servidor pueda diferenciar a cada cliente (por ejemplo: IP + puerto o un nickname).
- Los mensajes enviados por un cliente se distribuyan a los demás (tipo chat).

Servidor

1. Crear un socket TCP que escuche en un puerto (ej. 5000).
2. Por cada nueva conexión:
    - Aceptar con `accept()`.
    - Crear un nuevo thread para manejar ese cliente.
3. Mantener una estructura compartida (lista o diccionario) con:
    - Identificador del cliente (IP:puerto o nickname).
    - Socket asociado.
4. Implementar demultiplexación lógica:
    - El servidor debe identificar de qué cliente proviene cada mensaje.
    - Mostrar en consola algo como:

      ```
      [Cliente 192.168.1.5:53000]: Hola
      ```
5. Reenviar mensajes a los demás clientes conectados.
6. Manejar desconexiones:
    - Eliminar al cliente de la lista.
    - Notificar al resto.

Cliente

1. Conectarse al servidor.
2. Enviar un nickname al inicio.
3. Debe poder enviar y recibir mensajes ¿Es necesario crear dos threads?
4. Mostrar mensajes en formato:

```
[Juan]: Hola a todos
```

Ejemplo de ejecución

Servidor:

```
Servidor escuchando en puerto 5000...
Cliente conectado: 127.0.0.1:54021 (Juan)
Cliente conectado: 127.0.0.1:54022 (Ana)

[Juan]: Hola
[Ana]: Hola Juan!
```

Cliente:

```
Conectado al servidor
Ingresa tu nombre: Juan

[Servidor]: Ana se ha unido
[Ana]: Hola!
```